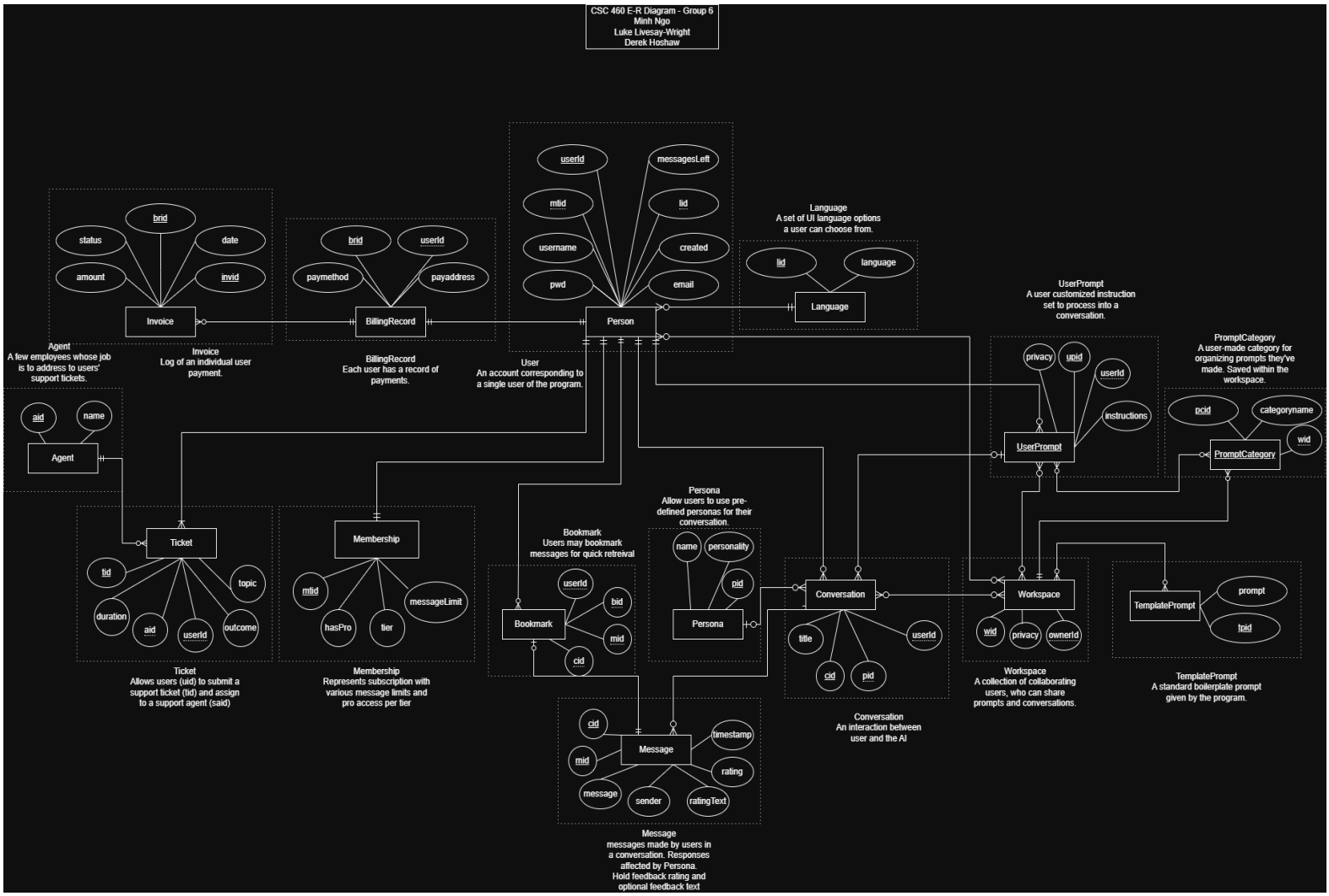
Minh Ngo
Luke Livesay-Wright
Derek Hoshaw
Group 6
CSC 460
05/05/2026

Project 4 Design Writeup

a) Conceptual Database Design

Your final E–R diagram along with your design rationale and any necessary high–level text description of the data model (e.g., constraints, or anything you were not able to show in the E–R diagram but that is necessary to help people understand your database design)



For our E-R diagram we decided to centralize everything around the **User** entity. Each feature branches in different directions with the necessary entities to achieve the correct functionality.

For the **User** entity, we have attributes (as listed in spec) for name, date created, and email. We also added primary/foreign keys for UserID (userId), languageID (lid) to identify the users preferred UI language, and the membershipTierID (mtid) to identify what subscription tier the user has.

The main functionality comes from the **Message** and **Conversation** entities. With a ConversationID (cid) to identify unique conversations and to identify which messages belong to the same conversation. Each message has a distinguishing key MessageID (mid) with the necessary attributes (as per spec) for the message, timestamp, the sender's role (User vs AI) and space for feedback rating and text.

Users can select predefined Persona's for their conversations via the **Persona** entity linked to the **Conversation** entity. Users can also bookmark specific messages through the **Bookmark** entity where the bookmarkID (bid) is paired with a messageID (mid) for quick retrieval.

There is also the feature for user workspaces. For this, we added a **Workspace** entity which links various users who can have the same prompts and conversations. For prompts, we have three entities. **UserPrompt** for the prompt the user selects on their conversation, **PromptCategory** to organize the UserPrompt entities, as well as **TemplatePrompt** to give the user premade prompts they can use in their workspace.

On the other side of the E-R diagram we have the other features of the program including support tickets, memberships, and billing. For support tickets, users can call a **Ticket** entity which keeps attributes for the ticketID (tid), the duration of the ticket, the outcome, and the topic, as well as a shared **Agent** entity (aid) for a Support Agent which can be called to handle each ticket.

For memberships/subscriptions, we have the **Membership** entity which has a membershipTierID (mtid), as well as the name of the tier, the message rate limit for the tier (to check if the user has exceeded their rate limit), and a binary attribute to see if the user has access to pro models.

For billing, users can access the **BillingRecord** entity which has a BillingRecordID (brid), the payment method, and payment address. Within that entity, there is an additional entity for monthly invoices. The **Invoice** entity contains an identifier (invid), the date of invoice, the amount due, and the status of whether the user has paid or not.

NOTE: Because "User" is a blocked keyword on Oracle, we used "Person" as a substitute entity name within our code.

b) Logical Database Design

The conversion of your E–R schema into a relational database schema. Provide the schemas of the tables resulting from this step.

Person	**userId**	**mtid**	username	created	email	pwd	**lid**
Conversation	**cid**	title	**userId**	**pid**			
Message	**mid**	**cid**	message	sender	ratingText	rating	timestamp
Bookmark	**bid**	**mid**	**userId**	**cid**			
Persona	**pid**	name	personality				
Workspace	**wid**	privacy	ownerId				
TemplatePrompt	**tpid**	prompt					
PromptCategory	**pcid**	categoryname	**wid**				
UserPrompt	**upid**	instructions	privacy	**userId**			
Language	**lid**	language					
Membership	**mtid**	hasPro	tier	limit			
Ticket	**tid**	duration	outcome	topic	**aid**	**userId**	
Agent	**aid**	name					
BillingRecord	**brid**	paymethod	payaddress	**userId**			
Invoice	**invid**	amount	status	date	**brid**		

NOTE: Each row represents a unique entity set (table).

Extra Tables: These are needed to separate Many-to-Many Relationships based on our E-R diagram.

UserWorkspace	userId	wid
UserPromptWorkspace	userId	wid
TemplatePromptWorkspace	wid	tpid
PromptCategoryUserPrompt	pcid	upid

ConversationWorkspace	cid	wid

c) Normalization Analysis

For each of your entity sets (tables), provide all of the FDs of the table and justify why your table adheres to 3NF / BCNF.

User

userId → mtid, username, created, email, pwd, lid
X username → userId , mtid, created, email, pwd, lid
X email → userId , mtid, created, username, pwd, lid

Conversation

cid → title, userId, pid

Message

mid → cid, message, sender, ratingText, rating, timestamp

Bookmark

bid → mid, userId
(userId, mid) → bid

Persona

pid → name, personality, userId

Workspace

wid → privacy

TemplatePrompt

tpid → prompt

PromptCategory

pcid → categoryname

UserPrompt

upid → instructions, privacy, userId

Language

lid → language

Membership

mid → hasPro, tier, limit

Ticket

tid → duration, outcome, topic, aid, userId

Agent

aid → name

BillingRecord

brid → paymethod, payaddress, userId

Invoice

invid → amount, status, date, brid

Each entity set in our database design adheres to 3NF/BCNF because there are no partial dependencies (2NF), no transitive dependencies (3NF), and they all have identifying superkeys such as conversationID (cid) or agentID (aid) which satisfies BCNF.

For all Extra tables (i.e. UserWorkspace, UserPromptWorkspace, etc.), since they strictly only hold foreign keys to create a composite key, there exists no non-prime attributes, and therefore is vacuously in 3NF.

d) Query Description

Describe your self–designed query. Specifically, what question is it answering, and what is the utility of including such a query in the system?

For our self-designed query, we wanted to find the most active users in our system. For this query, we are searching for the Users with the top 5 most messages sent. Then, returning their username, membership tier, the total number of messages they have sent, the average message length, and other identifying information. If this were an

actual AI system, having this utility could help us identify how our most active users are using our system and how we can better improve.